

ComunikApp - Especificação Técnica - Módulo de Produtos Finitos

Público-alvo: Cursor AI / Desenvolvedor Core

Encoding Mandatório: UTF-8 (Strict) com acentuação correta pt-br

Objetivo: Implementar a área de gestão de produtos acabados de prateleira/revenda (venda por unidade), integrando-os ao ecossistema de orçamentos e estoque sem romper o modelo multi-tenant atual.

1. Diretriz de Segurança de Dados e Migração Imutável

Para garantir risco zero de perda de dados históricos de clientes e cadastros vigentes durante o deploy, a migração do banco de dados deve obedecer estritamente aos seguintes critérios:

- **Não-destrutividade:** É expressamente proibido alterar, truncar ou dropar tabelas existentes como insumos, produtos (antiga/templates) ou orcamentos.
- **Isolamento de Entidades:** O novo modelo de produtos finitos deve residir em tabelas independentes, estabelecendo relações via chaves estrangeiras seguras e opcionais se tocarem em tabelas legadas.
- **Deploy Seguro:** Toda alteração de schema deve ser validada localmente e aplicada via comandos que executam migrações incrementais (npx prisma migrate deploy), sem invocar resets de banco.

2. Modelo de Dados (Prisma Schema)

Adicione os seguintes blocos ao arquivo backend/prisma/schema.prisma. Certifique-se de manter o padrão multi-tenant através do campo loja_id.

```
model CategoriaProdutoFinito {
  id      String      @id @default(uuid())
  loja_id String      // Isolamento Multi-tenant (padrão loja_id)
  nome    String      @db.VarChar(100)
  ativo   Boolean      @default(true)
  criado_em DateTime   @default(now())
}
```

```
    atualizado_em DateTime @updatedAt
    produtos ProdutoFinito[]
```

```
    @@index([loja_id])
    @@map("categorias_produto_finito")
}
```

```
model ProdutoFinito {
  id String @id @default(uuid())
  loja_id String // Isolamento Multi-tenant
  categoria_id String?
  sku String @db.VarChar(50) // Código identificador para e-commerce
  ean String? @db.VarChar(13) // Código de barras para marketplaces
  nome String @db.VarChar(255)
  descricao String? @db.Text
  preco_venda Decimal @db.Decimal(10, 2)
  preco_promocional Decimal? @db.Decimal(10, 2)
  peso_kg Decimal @db.Decimal(10, 3) @default(0.000)
  largura_cm Int @default(0)
  altura_cm Int @default(0)
  profundidade_cm Int @default(0)
  estoque_atual Int @default(0)
  estoque_minimo Int @default(0)
  ativo Boolean @default(true)
  criado_em DateTime @default(now())
  atualizado_em DateTime @updatedAt
```

```
  categoria CategoriaProdutoFinito? @relation(fields: [categoria_id], references: [id],
onDelete: SetNull)
  imagens GaleriaProdutoFinito[]
```

```
    @@unique([loja_id, sku]) // SKU único por loja
    @@index([loja_id])
    @@map("produtos_finitos")
}
```

```
model GaleriaProdutoFinito {
  id String @id @default(uuid())
  produto_finito_id String
  url_imagem String @db.Text
  ordem Int @default(0)
  produto ProdutoFinito @relation(fields: [produto_finito_id], references: [id], onDelete:
Cascade)
```

```
    @@map("galeria_produto_finito")  
  }
```

3. Arquitetura do Backend (NestJS)

Crie um novo módulo chamado ProdutoFinitoModule. Ele deve expor endpoints REST protegidos pelo middleware global de autenticação JWT, extraindo o loja_id do token do usuário logado.

Método	Endpoint	Descrição
GET	/produtos-finitos	Lista produtos finitos da loja (com paginação e filtro por categoria).
POST	/produtos-finitos	Cadastra novo produto finito. Valida SKU único e injeta o loja_id.
PUT	/produtos-finitos/:id	Atualiza dados cadastrais, dimensões e estoque do produto.
DELETE	/produtos-finitos/:id	Soft delete (ou altera flag ativo para false) para preservar histórico.

4. Estratégia de Integração com Orçamento V2

Para viabilizar a inclusão de produtos de prateleira na tela de Orçamentos V2, modifique o fluxo de itens do orçamento:

- Discriminação do Tipo de Item:** No DTO de entrada de itens do orçamento, adicione um campo diferenciador: tipo_item: 'SOB_DEMANDA' | 'PRODUTO_FINITO'.
- Bypass no Motor de Cálculo V2:** Se o item for PRODUTO_FINITO, o MotorCalculoV2Service não computará metros quadrados, perdas, insumos ou tempos

de máquina. O preço final do item será a multiplicação direta: preco_venda (ou preco_promocional) $\times$ quantidade.

3. **Persistência no Orçamento:** Salve o ID do produto finito na relação do item para que o módulo de estoque saiba exatamente o que decrementar após a aprovação comercial.

5. Especificação do Frontend (Next.js 15 App Router)

- ---

Nova Rota Comercial: Criar as interfaces sob a pasta `src/app/(dashboard)/produtos-finitos/page.tsx` para listagem e gerenciamento.
- **Formulário de Cadastro Estruturado:** Organizar os inputs em seções visuais limpas utilizando Tailwind CSS:
 - *Identificação Comercial:* Nome, Descrição, SKU, EAN/GTIN e Categoria.
 - *Precificação:* Preço de Venda e Preço Promocional (Inputs com máscara monetária).
 - *Dados de Logística e Frete (Essencial para os futuros Marketplaces):* Peso (kg), Altura (cm), Largura (cm), Profundidade (cm) da embalagem de envio.
 - *Mídia:* Upload múltiplo de fotos gerenciando a ordenação das imagens na tabela de galeria.
- **Renomeação de Rotas Legadas:** Modificar os menus e rotas do antigo "Produtos" para "Modelos de Orçamento" ou "Templates", alterando apenas as labels e caminhos visuais para refletir o novo padrão sem quebrar as tabelas de banco correspondentes.